גיליון תשובות

מספר נבחן:_____

שאלה 1: זיכרון (35 נקודות)

א-ב.

```
template <typename T>

class BinaryTree {

private:
    T data;
    BinaryTree<T>* left;
    BinaryTree<T>* right;

    void clean() {
        if (left != nullptr) {
            delete left;
            left = nullptr;
        }
        if (right != nullptr) {
            delete right;
            right = nullptr;
        }
    }

    void copy(const BinaryTree& other) {
        if (other.left == nullptr)
            left = nullptr;
        else
            left = new BinaryTree<T>(*(other.left));
        if (other.right == nullptr)
            right = nullptr;
```

```
        else
            right = new BinaryTree<T>(*(other.right));
    }

public:

    BinaryTree(T d) :  data(d), left(nullptr), right(nullptr) {}
    BinaryTree(T d, BinaryTree<T>* l, BinaryTree<T>* r) :
       data(d), left(l), right(r) {}

    BinaryTree(const BinaryTree<T>& other) : data(other.data) {
        copy(other);
    }

    ~BinaryTree() { clean(); }

    BinaryTree<T>& operator=(const BinaryTree<T>& other) {
        if (this != &other) {
            data = other.data;
            clean();
            copy(other);
        }
        return *this;
    }

    BinaryTree(BinaryTree<T>&& other) :
        data(other.data), left(other.left), right(other.right) {
        other.left = nullptr;
        other.right = nullptr;
    }

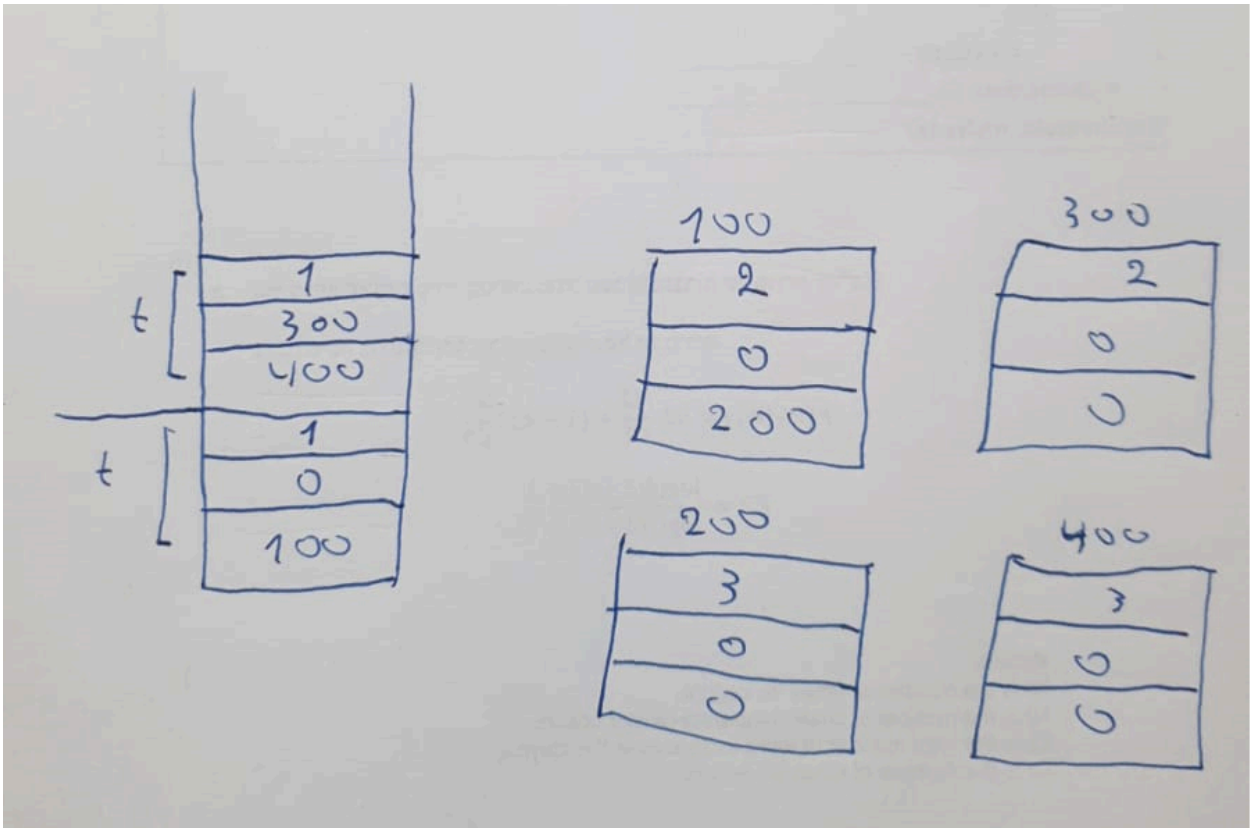
    BinaryTree<T>& operator=(BinaryTree<T>&& other) {
        if (this != &other) {
            data = other.data;
            clean();
            left = other.left;
            right = other.right;
```

```
            other.left = nullptr;
            other.right = nullptr;
        }
        return *this;
    }

    T getData() const { return data; }
    BinaryTree<T>& getLeft() const { return *left; }
    BinaryTree<T>& getRight() const { return *right; }

    void insert(T d) {
        if (data == d)
            return;
        if (data > d) {
            if (left == nullptr)
                left = new BinaryTree<T>(d);
            else
                left->insert(d);
        } else {
            if (right == nullptr)
                right = new BinaryTree<T>(d);
            else
                right->insert(d);
        }
    }
};
```

ג.



- ציור של t (השדות שלו) על Heap ולא על המחסנית, או באופן כללי אי שמירת שדות על המחסנית או על Heap כאשר היה צורך. (2-).
- פתרון/ציור חלקי רק של שורות 1-3, או רק קריאה לפונ f והתעלמות מAF קודמים. (5-).
- לא צוין ערך/שם השדה (1-)
- קיום של other(ביטוי rvalue) במחסנית ושימוש ב copy constructor. הפחתה של 3 נק.
- כן התקבלו תשובות אשר בהן סטודנטים מיינו את הקוד׳ של העץ ולא הכניסו אותם בדיוק כפי שהתקבלו בבנאי.

שאלה 2: מקביליות (30 נקודות)

א. נוסיף לממשק את השאילתות הבאות:

```
int size()
T itemAt(int i)
int occurs(T item)  // returns the number of occurrences of a given item

@INV: for 0 <= i < size(), occurs(itemAt(i)) == 1

@PRE: occurs(@param obj) == 0
@POST: occurs(@param obj) == 1
```

בדיקה:
טעות נפוצה היתה שלא לבטא באינווריאנטה את התכונה המרכזית של סט - מבנה נתונים עם איברים ייחודיים (עד 4 נק').

ב. המחלקה בטוחה תחת כל חישוב מקבילי
האינווריאנטה נשברת אם אובייקט מוכנס פעמים לקבוצה. זה אפשרי לכאורה אם שני ת'רדים מבצעים בו זמנית add על אותו אובייקט, אך בקוד הנתון זה לא יקרה טכנית, כי או שהת'רד הראשון מספיק לבצע את ההשמה במתודה add (השורה השניה להלן) לפני שהת'רד השני בדק את התנאי (השורה הראשונה להלן), או שהת'רד הראשון ביצע את ההשמה אחרי שהת'רד השני בדק את התנאי, ואז הת'רד השני ידרוס את ההשמה של הת'רד הראשון:

```
if (curr.next == null)
            curr.next = new Link(obj,null);
```

אין בעיות visibility כי החוליות הן **.volatile**

בדיקה:
טעות נפוצה - תשובה ללא נימוק. המשפט "המחלקה בטוחה כי היא שומרת על האינווריאנטה ותמיד מתקיים X Y" אינו נחשב נימוק, אלא חזרה על הגדרת הבטיחות. תשובות כנ"ל קיבלו 1 נק' בד"כ. תשובה מלאה אמורה לכלול התייחסות לגישה מקבילית על ידי ת'רדים מרובים, ובפרט לדריסה של חוליות על ידי ת'רדים.
טעות נפוצה נוספת - שגיאה נגררת מסעיף א'. למשל סטודנטים שציינו אינווריאנטה שגויה או חסרה (למשל שתמיד יש לפחות 0 איברים במבנה הנתונים), ונימקו איך האינווריאנטה השגויה מתקיימת. תשובות כנ"ל קיבלו בד"כ 1 נק', כמו עבור תשובות לא מנומקות. הסיבה - תכונות כגון "יש לפחות 0 איברים במבנה הנתונים", או "אין תכונה נשמרת" - נשמרות באופן טריוויאלי. סטודנט שפתר כך, למעשה פתר שאלה קלה יותר מזו שהתכוון אליה כותב המבחן.

ג. המחלקה אינה נכונה תחת כל חישוב מקבילי

תרחיש: ת'רד אחד מוסיף לקבוצה ריקה 1 ות'רד שני 2. בהרצה סדרתית בסדר כלשהו יהיו בקבוצה בסוף גם 1 וגם 2. בהרצה מקבילית ייתכן ששני הת'רדים בדקו את התנאי על head וראו שהוא null, ואחר כך בצעו בזה אחר זה השמה ל-head עם הערך שלהם, כך שאחד הערכים נדרס ולא ייכלל בקבוצה.

```
if (head == null)
            head = new Link<T>(obj,null);
```

ג.

```
class SetImpl<T> implements Set<T> {

    private AtomicReference<Link<T>> head;

    SafeImpl() {
```

```
        head = new AtomicReference<Link<T>>(null);
    }

    public boolean contains(T obj) {
       ...
    }

    public void add(T obj) {
        if (!head.compareAndSet(null, new Link<T>(obj,null)))
            add(head.get(),obj);
    }

    private void add(Link<T> curr, T obj) {
        if (curr.data.equals(obj))
            return;
        if (!(new AtomicReference<Link<T>>(curr.next)).compareAndSet
          (null, new Link<T>(obj,null)))
            add(curr.next,obj);
    }

    ...
}
```

פתרון המבוסס על busy-wait עם cas על משתנה בוליאני אינו lock-free, כפי שנלמד במפורש בהרצאה.

שאלה 3: תקשורת ובסיסי נתונים (35 נקודות)

א. הצעת הסטודנט פותח אפשרויות מקביליות שלא היו בקוד המקורי:

- שני ת'רדים יכולים לבצע במקביל continueRead על אותו לקוח
- שני ת'רדים יכולים לבצע במקביל continueWrite על אותו לקוח
- ת'רד אחד יכול לבצע continueRead על לקוח במקביל לת'רד שני המבצע continueWrire על אותו לקוח
- ת'רד עבודה מבצע task של לקוח בזמן שת'רד תקשורת, שאינו הת'רד המבצע selector.select, מבצע במקביל פעולת continueWrite/Read על אותו לקוח

לאור זאת ייתכנו לפחות חמשת התרחישים הבאים:

(i-ii) שני ת'רדים מבצעים continueRead עבור אותו לקוח. הקריאה מה-channel נעשית אמנם באופן תקין, כי SocketChannel בטוחה ונכונה, אך ייתכן שה-tasks שנוצרו על בסיס מערכי הבתים שנקראו הוכנסו לתור

המשימות ב-ActorThreadPool של ת'רדי העבודה בסדר הפוך.
זה עשוי לגרום לשתי בעיות:

- הבתים מגיעים לאנקודר של הלקוח כחלקי הודעות בסדר לא נכון המערבב בתים מהודעות שונות, כך שמתבצעות הודעות שלא נשלחו.

- הבתים מגיעים לאנקודר של הלקוח כהודעות שלמות בסדר לא נכון, כך שמתבצעות ההודעות שהלקוח שלח, אך בסדר הפוך.

(iii-iv) שני ת'רדים מבצעים continueWrite עבור אותו לקוח.

הכתיבה ל-channel נעשית באופן תקין:

- SocketChannel בטוחה ונכונה.
- פעולת הכתיבה ל-SocketChannel מסירה את הבית הנכתב מה-ByteBuffer, כך שלא ייתכן שאותו בית ייכתב פעמיים.
- גם לא ייתכן שיש לכל ת'רד ByteBuffer שונה וההודעות לא יישלחו על פי הסדר, כי אנו מסירים תשובה מהתור רק לאחר שכולה נכתבה.

אך ייתכנו בעיות נוספות:

- שני התת'רדים נגשים עם peek לאותו ByteBuffer, הראשון כותב, השני כותב כלום כי הבתים נכתבו כבר, ואז שניהם מבצעים remove כך שהתשובה הבאה נמחקת לפני שנשלחה.

- ת'רד אחד מבצע **`chan.write(top()`** במקביל לת'רד אחר שמבצע **`top.hasRemaining()`**. מאחר ש-ByteBuffer אינה בטוחה ונכונה, תיתכן תוצאה שגויה.

(v) ת'רד עבודה עובד במקביל לת'רד תקשורת המבצע continueWrite על אותו לקוח. ת'רד התקשורת כותב את כל התשובות ומעביר משימה לשינוי רישום ל-OP_READ (הוא לא הת'רד של הסלקטור אז זה לא מתבצע בפועל), אך בין הבדיקה שהרשימה ריקה לבין הוספת הטאסק של שינוי הרישום, ת'רד עבודה הוסיף בקשה לשינוי רישום ל- OP_READ | OP_WRITE. כשהת'רד של הסלקטור יבצע סדרתית משימות אלו, הוא ישנה את רישום ערוץ הלקוח ל-OP_READ | OP_WRITE אך אח"כ יבצע את משימת שינוי הרישום ל-OP_READ בלבד, יחזור לישון על תנאי זה והתשובה לא תישלח ללקוח.

<u>הערה חשובה לפני שמערערים</u>
היה מצופה במבחן שתציינו בעיות מהותיות הנובעות מעדכון הקוד:
- בעיות של 'חוסר הוגנות' אינן רלבנטיות, בין אם מדובר בשני ת'רדים העובדים על צ'אנל של אותו לקוח (לא משנה איזה מהם רץ), בין אם ת'רדים העובדים על לקוחות שונים (תזמון לא הוגן יכול להיות גם בקוד המקורי בריאקטור, כאשר יש ת'רד תקשורת יחיד שמטפל סדרתית באירועים, הוא עשוי לדוגמא לא לקבל זמן מעבד כשלקוחות מסויימים שולחים הודעות, ולקבל זמן מעבד ברגע שלקוח אחר שנמצא לפניו בסלקטור שולח הודעה)
- בעיות של 'ביצוע משימה ריקה' וכדומה אינן נחשבות לבעיות קבילות, כך גם עבור שלל 'בעיות' שוליות שנכתבו בתשובות

נתתי לא פעם ניקוד חלקי על כל מיני דברים לא נכונים, בבדיקת הערעור הניקוד הזה יישקל בשנית.

ב. בעיות i-iv נפתרות בפשטות על ידי שינוי הטיפוס של ה-communicationThreads ל-ActorThreadPool. אך זה לא מספיק לבעיה החמישית, כי ת'רד העבודה ות'רד התקשורת נמצאים ב-ActorThreadPools שונים, ולכן בנוסף לעדכון הטיפוס של ה-communicationThreads ל-ActorThreadPool יש לסנכרן את ת'רד העבודה

ות'רד התקשורת על ה-writeQueue בקטעי הקוד הקריטיים ב-continueWrite ובמשימת ת'רד העבודה המוגדרת ב-continueRead:

```
class Reactor {

  ActorThreadPool<NonBlockingConnectionHandler> communicationThreads =
         new ActorThreadPool<>(2);

  ...

 private void handleReadWrite(SelectionKey key) {
        @SuppressWarnings("unchecked")
        NonBlockingConnectionHandler<T> handler =
                    (NonBlockingConnectionHandler<T>) key.attachment();
        if (key.isReadable()) {
            communicationThreads.submit(handler, () -> {
               Runnable task = handler.continueRead();
               if (task != null)
                   pool.submit(handler, task);
               else if (!key.isValid())
                   return;
            });
        }
        if (key.isWritable()) {
           communicationThreads.submit(handle, () -> {
               handler.continueWrite();
           });
        }
    }

    ...
}

class NonBlockingConnectionHandler<T>  implements ConnectionHandler<T> {{

...

public Runnable continueRead() {

    ...

    return () -> {
        try {
          while (buf.hasRemaining()) {
            T nextMessage = encdec.decodeNextByte(buf.get());
            if (nextMessage != null) {
              T response = protocol.process(nextMessage);
              if (response != null) {
```

```
                synchronized (writeQueue) {
                 writeQueue.add(ByteBuffer.wrap(
                                    encdec.encode(response)));
                 reactor.updateInterestedOps(chan, SelectionKey.OP_READ
                                                 | SelectionKey.OP_WRITE);
                }
               }
              }
             }
           } finally {
             releaseBuffer(buf);
           }
         };

       ...

     }


   public void continueWrite() {
     while (!writeQueue.isEmpty()) {
       try {
         ByteBuffer top = writeQueue.peek();
         chan.write(top);
         if (top.hasRemaining()) {
           return;
         } else {
           writeQueue.remove();
         }
       } catch (IOException ex) {
         ex.printStackTrace();
         close();
       }
     }
     synchronized (writeQueue) {
       if (writeQueue.isEmpty()) {
         if (protocol.shouldTerminate()) close();
           else reactor.updateInterestedOps(chan, SelectionKey.OP_READ);
       }
     }
   }

...

}
```

על פתרון שלא כלל סנכרון של ה-writeQueue ב-continueWrite ירדו שתי נקודות.

על פתרון בו נעשה שימוש ב-ActorThreadPool יחיד ירדו שתי נקודות - החיסרון בפיתרון זה הוא שאין הבחנה בין כמות הת'רדים המוקדשת לתקשורת וכמות הת'רדים המוקדשת לעבודה:

```
public class Reactor<T> implements Server<T> {

    ~~ExecutorService communicationThreads = Executors.newFixedThreadPool(2);~~

    ...

    public Reactor(
            int numThreads,
            int port,
            Supplier<MessagingProtocol<T>> protocolFactory,
            Supplier<MessageEncoderDecoder<T>> readerFactory) {

        this.pool = new ActorThreadPool<>(numThreads + 2);
        this.port = port;
        this.protocolFactory = protocolFactory;
        this.readerFactory = readerFactory;
    }

    ...

    private void handleReadWrite(SelectionKey key) {
        @SuppressWarnings("unchecked")
        NonBlockingConnectionHandler<T> handler =
                    (NonBlockingConnectionHandler<T>) key.attachment();
        if (key.isReadable()) {
            pool.submit(handler, () -> {
               Runnable task = handler.continueRead();
               if (task != null)
                   pool.submit(handler, task);
               else if (!key.isValid())
                   return;
            });
        }
        if (key.isWritable()) {
           pool.submit(handler, () -> {
               handler.continueWrite();
           });
        }
    }
```

ג. הצעת הסטודנטית אינה פותרת את בעיה (v) שתוארה בסעיף א, הנובעת ממקביליות של ת'רד עבודה ותקשורת שאינו ה-selectorThread.

תשובה שלא עמדה על כך (גם במקרה שלא מצאתם בעיה זו בסעיף א) הינו שגויה. בכוונה לא התייחסתי לכך כטעות נגררת, מדובר בבעייה בסיסית הקשורה להבנת מנגנון הריאקטור. הפרדתי מראש את הסעיף הזה מסעיף א כדי לתת יותר אפשרות לניקוד מלא על סעיף א, גם למי שלא עמד על הבעיה שבסעיף ג.